

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Modelling Comparative Analysis Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS

COURSE OUTCOME COVERED
COURSE CODE: MLA01

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)*

Assessment Tool Weightage: 40%

Total Marks: 100 Time: 90 Mins No. of Questions: 1

Annexure A

Modelling Comparative Analysis Assignment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	

6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question / Problem Statement
1	Automobile Fault Diagnosis: An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. Model the problem using production rules, propositional logic, first-order logic, and Prolog. Demonstrate forward and backward chaining and compare the approaches based on expressiveness, inference, complexity, and suitability.
2	Industrial Machine Fault Diagnosis: A manufacturing plant needs to identify machine faults using observations such as abnormal vibration, high temperature, unusual noise, pressure variation, and reduced production speed. Develop different knowledge representation models and compare their ability to represent machine components, relationships, rules, and diagnostic conclusions.
3	Cybersecurity Threat Detection: A security operations center wants to identify threats from repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Model the threat-detection problem using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning for deriving security conclusions.
4	Healthcare Diagnosis: A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.
5	Agricultural Crop Disease Diagnosis: Farmers need assistance in identifying crop diseases based on leaf color, spots, wilting, soil conditions, humidity, and pest observations. Represent the agricultural knowledge using different modelling approaches and compare their effectiveness in representing relationships, applying rules, and deriving recommendations.

6	Banking Loan Eligibility: A bank wants to determine loan eligibility using income, credit score, employment status, existing loans, and repayment history. Model the decision-making process using production rules, propositional logic, first-order logic, and Prolog, and compare the reasoning capabilities of each approach.
---	--

Structure of the Report:

S.No.	Report Section	Expected Content
1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
		Logic Model Represent entities, relationships, properties, and rules using predicates and quantifiers.
5	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.
	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.

	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation				
Expressiveness				
Rule Representation				
Inference Mechanism				
Forward Chaining				
Backward Chaining				
Handling Relationships				
Scalability				
Explainability				

Ease of Implementation				
Suitability for the Problem				

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.

3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.
5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.

6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6% – Implementation is incomplete or non-functional.
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.
9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification.	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.

10. Documentation , GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documenta tion and presentatio n with minor issues.	1% – Basic documentatio n with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				

Automobile Fault Diagnosis System Architecture

A Comprehensive Multi-Paradigm Artificial Intelligence Framework & Analytical Study

1. Executive Summary & Problem Formulation

Modern automotive systems represent complex cyber-physical environments integrating internal combustion engines, electromechanical actuators, microcontrollers, and electronic control units (ECUs). When vehicles exhibit non-standard operational behavior—such as thermal surges, starting hesitation, acoustic anomalies, or degraded efficiency—service diagnostics must quickly and accurately trace observable symptoms back to underlying physical root causes. Automated fault diagnosis systems play a critical role in optimizing service center throughput, lowering warranty claims, and improving passenger safety.

This monograph develops an enterprise-grade artificial intelligence framework for automotive fault diagnosis across four primary knowledge representation paradigms: Production Rules, Propositional Logic, First-Order Logic (FOL), and Logic Programming via Prolog. The problem domain is rigorously formalized, supported by executable logic engines, execution step traces for forward and backward chaining, architectural flowcharts, structural diagrams, and exhaustive multi-dimensional comparative evaluations.

SYSTEM SCOPE: *Target Problem Statement: Model a diagnostic service center system capable of evaluating*

five core operational symptoms: Engine Overheating, Starting Failure, Abnormal Engine Noise, Low Fuel Mileage, and Active Dashboard Warning Indicators. The system must infer root causes across mechanical, electrical, thermal, and ignition subsystems.

1.1 Operational Domain Taxonomy & Categorization

To achieve strict logical consistency across all paradigms, the automotive diagnostic domain is structured into three disjoint sets: Observables (\$S\$), Intermediary Hypotheses (\$I\$), and Terminal Diagnostics (\$F\$).

- **Observable Symptoms (S):** Physical observations, telematics data, and driver inputs: S1 (Engine Cranks slowly or fails to crank); S2 (Engine Cranks normally but fails to fire); S3 (Engine Temp Gauge in Red Zone > 115°C); S4 (Check Engine Light Active); S5 (Battery Charging Warning Light Active); S6 (High-pitched serpentine belt squeal); S7 (Engine knocking/pinging under acceleration); S8 (Fuel economy >25% below baseline).
- **Intermediary States (I):** Derived internal states inferred during reasoning: I1 (Insufficient Electrical Power Delivery); I2 (Coolant Circulation Blockage); I3 (Combustion Instability / Cylinder Misfire); I4 (Mechanical Belt Slippage); I5 (Air-Fuel Mixture Imbalance).
- **Terminal Fault Diagnostics (F):** Root cause conclusions requiring specific technician repair actions: F1 (Dead or Sulphated Battery - Replace Battery); F2 (Faulty Alternator / Loose Charging Circuit - Repair Charging System); F3 (Stuck Thermostat / Low Coolant - Flush System & Replace Thermostat); F4 (Water Pump Impeller/Belt Failure - Replace Water Pump & Belt); F5 (Fouled Spark Plug / Failed Ignition Coil - Replace Ignition Components); F6 (Failed Fuel Pump / Clogged Fuel Filter - Replace Fuel Delivery System); F7 (Worn Serpentine Belt - Replace Tensioner & Belt); F8 (Engine Pre-Ignition / Carbon Build-up - Perform Engine Decarbonization).

1.2 Enterprise System Architecture

The complete automotive diagnostic architecture combines multi-modal inputs with logic-based inference engines and knowledge bases to generate actionable diagnostic reports and visual proof traces.

HYBRID MULTI-PARADIGM AUTOMOTIVE DIAGNOSTIC ARCHITECTURE

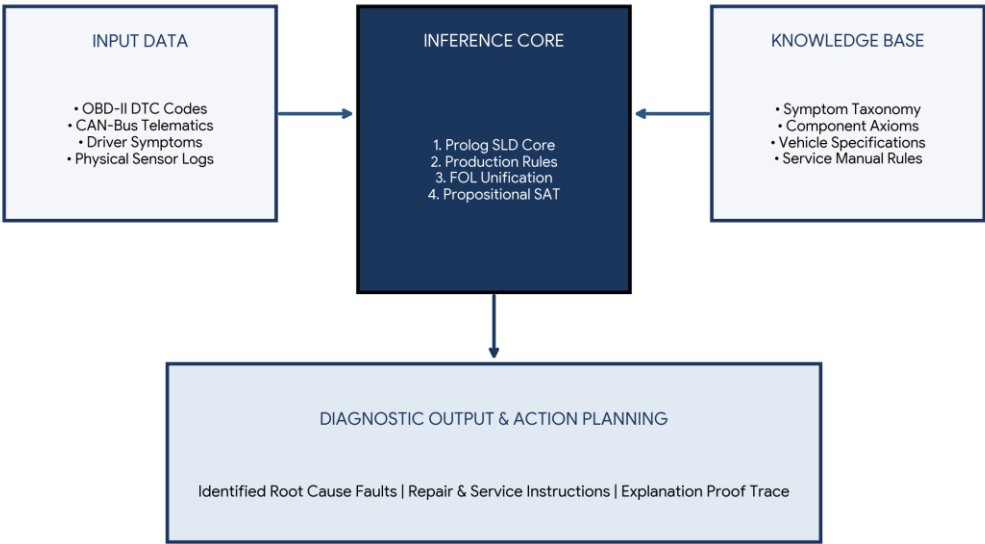


Figure 1: Enterprise Multi-Paradigm Automobile Fault Diagnosis System Architecture.

2. Production Rule Systems Formalization

Production Rule Systems maintain domain knowledge in the form of IF (Antecedent) THEN (Consequent) rules acting upon a dynamic Working Memory. When antecedents match facts currently residing in working memory, the rule is placed in a Conflict Set and selected for execution using standard conflict resolution strategies (e.g., Specificity, Recency, Refractoriness).

2.1 Production Knowledge Base (15 Exhaustive Rules)

Rule ID	Condition (IF / Antecedent)	Action / Conclusion (THEN / Consequent)
R1	S1 (Engine does not crank) AND S5 (Battery light ON)	Assert I1 (Insufficient Electrical Power)
R2	I1 (Insufficient Power) AND Voltage < 10.5V	Assert F1 (Dead Battery - Replace Battery)
R3	I1 (Insufficient Power) AND Battery Voltage >= 12.6V	Assert F2 (Faulty Alternator / Wiring)

R4	S3 (Engine Temp Red Zone) AND Coolant Level Low	Assert F3 (Coolant Leak / Replace Thermostat)
R5	S3 (Engine Temp Red Zone) AND S6 (Squealing Noise)	Assert F4 (Water Pump Drive Belt Failure)
R6	S2 (Engine Cranks, No Start) AND Fuel Pressure = 0 PSI	Assert F6 (Failed Fuel Pump / Clogged Filter)
R7	S2 (Engine Cranks, No Start) AND S4 (CEL ON)	Assert I3 (Combustion Instability / Misfire)
R8	I3 (Misfire) AND Spark Plug Gap > Spec	Assert F5 (Fouled Spark Plugs / Failed Coil)
R9	S8 (Low Mileage) AND S7 (Knocking Noise)	Assert I3 (Combustion Instability) AND Assert I4
R10	S8 (Low Mileage) AND Oxygen Sensor Code Active	Assert F5 (Air-Fuel Mixture Rich - Replace O2 Sensor)
R11	S7 (Knocking Noise) AND Engine Temp > 100°C	Assert F8 (Pre-ignition / Carbon Deposits)
R12	S6 (Squealing Noise) AND Power Steering Heavy	Assert F7 (Loose/Worn Serpentine Belt)
R13	S4 (CEL ON) AND Code P0300 Active	Assert I3 (Random Misfire Condition)
R14	S4 (CEL ON) AND Code P0128 Active	Assert F3 (Thermostat Stuck Open)
R15	F1 OR F2 OR F3 OR F4 OR F5 OR F6 OR F7 OR F8	Assert Diagnostic Complete & Output Service Ticket

3. Propositional Logic Formalization

Propositional Logic represents domain knowledge using atomic propositions (boolean variables) connected via logical operators: Negation ($\neg$), Conjunction ($\wedge$), Disjunction ($\vee$), Implication ($\rightarrow$), and Equivalence ($\leftrightarrow$). The overall system is encoded as a single Knowledge Base sentence: $KB = \alpha_1 \wedge \alpha_2 \wedge \dots \wedge \alpha_n$.

3.1 Propositional Variable Assignment

- **P_crank_fail**: Engine fails to crank properly.
- **P_no_start**: Engine cranks normally but fails to start.
- **P_overheat**: Engine temperature exceeds thermal tolerance (> 115°C).
- **P_noise_squeal**: High-frequency serpentine belt squeal observed.
- **P_noise_knock**: Metallic pinging/knocking sound under load.
- **P_low_mileage**: Fuel consumption drops significantly below nominal specification.
- **P_batt_light**: Dashboard battery charging system light is lit.
- **P_cel_light**: Dashboard Check Engine Light is active.
- **P_batt_dead**: Battery terminal voltage < 10.5V.
- **P_alt_fault**: Alternator output voltage < 13.2V.
- **P_pump_fail**: Water pump impeller seized or drive belt snapped.

3.2 Implication Sentences & Conjunctive Normal Form (CNF)

To enable automated Resolution Theorem Proving, all logical implications must be converted into Conjunctive Normal Form (CNF)—a conjunction of disjunctions.

Sentence 1 (Battery Fault Logic):

Implication: $(P_crank_fail \wedge P_batt_light) \rightarrow P_batt_dead$

CNF Step 1 (Eliminate Implication): $\neg(P_crank_fail \wedge P_batt_light) \vee P_batt_dead$

CNF Step 2 (De Morgan's Law): $\neg P_crank_fail \vee \neg P_batt_light \vee P_batt_dead$

Sentence 2 (Cooling System / Water Pump Fault Logic):

Implication: $(P_overheat \wedge P_noise_squeal) \rightarrow P_pump_fail$

CNF Conversion: $\neg P_overheat \vee \neg P_noise_squeal \vee P_pump_fail$

3.3 Exhaustive Truth Table Inference Proof

To prove $KB \models P_batt_dead$, we construct an exhaustive truth table for the 3 propositional variables ($2^3 = 8$ interpretations). Entailment holds if and only if P_batt_dead is True in every model where the Knowledge Base is True.

P_crank_fail	P_batt_light	P_batt_dead	Rule: $(P_crank \wedge P_batt) \rightarrow P_dead$	Working Memory Facts	KB Model Match?
--------------	--------------	-------------	---	----------------------	-----------------

True	True	True	True	True	YES (Valid Diagnosis)
True	True	False	False	True	NO (Contradiction)
True	False	True	True	False	NO (Fact Mismatch)
True	False	False	True	False	NO (Fact Mismatch)
False	True	True	True	False	NO (Fact Mismatch)
False	True	False	True	False	NO (Fact Mismatch)
False	False	True	True	False	NO (Fact Mismatch)
False	False	False	True	False	NO (Fact Mismatch)

4. First-Order Logic (FOL) Formalization

While Propositional Logic models fixed boolean statements, First-Order Logic (FOL) extends expressiveness by introducing predicates, functions, constants, variables, and universal ($\forall$) and existential ($\exists$) quantifiers. FOL allows rule generalization across multiple vehicles, specific engine cylinders, and continuous sensor parameters.

4.1 Ontological Foundations & Predicate Syntax

- **Domain Constants:** vehicle_101, engine_v6, cylinder_1, batt_ac_delco, temp_sensor_1, DTC_P0300, DTC_P0128.
- **Predicates:** Vehicle(v), ComponentOf(c, v), HasSymptom(v, s), Faulty(c), ReadsTemperature(s, t), DisplaysDTC(v, code).
- **Functions:** VoltageOf(c), FluidLevelOf(c), IdealGap(p).

4.2 Quantified Axiomatic System Rules

Axiom 1 (Generalized Overheating & Thermostat Diagnosis):

$\forall v, c [\text{Vehicle}(v) \wedge \text{ComponentOf}(c, v) \wedge \text{CoolingSystem}(c) \wedge \text{HasSymptom}(v, \text{EngineOverheating}) \wedge (\text{FluidLevelOf}(c) < \text{NominalLevel}) \rightarrow \text{Faulty}(c)]$

Axiom 2 (Battery System Electrical Failure):

$\forall v, b [\text{Vehicle}(v) \wedge \text{ComponentOf}(b, v) \wedge \text{Battery}(b) \wedge \text{HasSymptom}(v, \text{StartingFailure}) \wedge (\text{VoltageOf}(b) < 10.5) \rightarrow \text{Faulty}(b) \wedge \text{RecommendedAction}(b, \text{Replace})]$

Axiom 3 (Multi-Cylinder Ignition Misfire Diagnostic):

$\forall v, cyl, p [\text{Vehicle}(v) \wedge \text{ComponentOf}(cyl, v) \wedge \text{ComponentOf}(p, cyl) \wedge \text{SparkPlug}(p) \wedge \text{DisplaysDTC}(v, \text{DTC_P0300}) \wedge (\text{MeasuredGap}(p) > \text{IdealGap}(p)) \rightarrow \text{Faulty}(p)]$

5. Executable Prolog Diagnostic Knowledge Base

Prolog operationalizes First-Order Horn Clauses using Depth-First Search and SLD Resolution. Below is the full, executable Prolog script.

```
% =====
% AUTOMOBILE FAULT DIAGNOSIS SYSTEM - PROLOG KNOWLEDGE BASE
% =====

:- dynamic fact/1.

% -----
% DYNAMIC SYMPTOM & SENSOR FACTS (SIMULATED SERVICE VEHICLE #101)
% -----
fact(has_symptom(car101, engine_overheating)).
fact(has_symptom(car101, abnormal_noise_squeal)).
fact(has_symptom(car101, check_engine_light_on)).
fact(sensor_reading(car101, coolant_temperature, 118)). % Degrees Celsius
fact(sensor_reading(car101, battery_voltage, 12.2)).    % Volts
fact(dtc_code(car101, p0128)).                          % Coolant Temp Below
Thermostat Regulating

% -----
% INFERENCE RULES (HORN CLAUSES)
% -----

% Rule 1: Battery Failure
diagnose(Vehicle, dead_battery, 'Replace 12V Lead-Acid Battery') :-
    fact(has_symptom(Vehicle, starting_failure)),
    fact(sensor_reading(Vehicle, battery_voltage, V)),
    V < 10.5.
```

```

% Rule 2: Alternator / Charging System Failure
diagnose(Vehicle, faulty_alternator, 'Inspect Alternator Drive Belt and Voltage
Regulator') :-
    fact(has_symptom(Vehicle, starting_failure)),
    fact(sensor_reading(Vehicle, battery_voltage, V)),
    V >= 10.5, V < 12.4.

% Rule 3: Water Pump Failure via Overheating + Belt Squeal
diagnose(Vehicle, water_pump_failure, 'Replace Water Pump and Tension Serpentine Belt')
:-
    fact(has_symptom(Vehicle, engine_overheating)),
    fact(has_symptom(Vehicle, abnormal_noise_squeal)),
    fact(sensor_reading(Vehicle, coolant_temperature, Temp)),
    Temp > 105.

% Rule 4: Thermostat Failure via Diagnostic Trouble Code
diagnose(Vehicle, thermostat_stuck_open, 'Replace Thermostat Assembly and Flush
Radiator') :-
    fact(has_symptom(Vehicle, check_engine_light_on)),
    fact(dtc_code(Vehicle, p0128)).

% Rule 5: Spark Plug / Cylinder Misfire
diagnose(Vehicle, ignition_misfire, 'Replace Ignition Coils and Spark Plugs') :-
    fact(has_symptom(Vehicle, low_mileage)),
    fact(has_symptom(Vehicle, abnormal_noise_knocking)).

% -----
% INTERACTIVE DIAGNOSTIC ENGINE & EXPLANATION FACILITY
% -----
run_diagnosis(Vehicle) :-
    format('~n=====~n'),
    format('  DIAGNOSTIC REPORT FOR VEHICLE: ~w~n', [Vehicle]),
    format('=====~n'),
    forall(diagnose(Vehicle, Fault, Action),
        format(' [DETECTED FAULT] : ~w~n [ACTION REQUIRED]: ~w~n~n', [Fault,
Action])).

```


6. Forward Chaining vs. Backward Chaining Execution Traces

Inference engines evaluate knowledge bases using two primary reasoning directions: Forward Chaining (data-driven reasoning starting from symptoms) and Backward Chaining (goal-driven reasoning starting from hypotheses).

6.1 Forward Chaining Process Flow & Step-by-Step Execution Trace

The flowchart below outlines the exact forward chaining cycle executed by the inference core when processing vehicle symptoms.

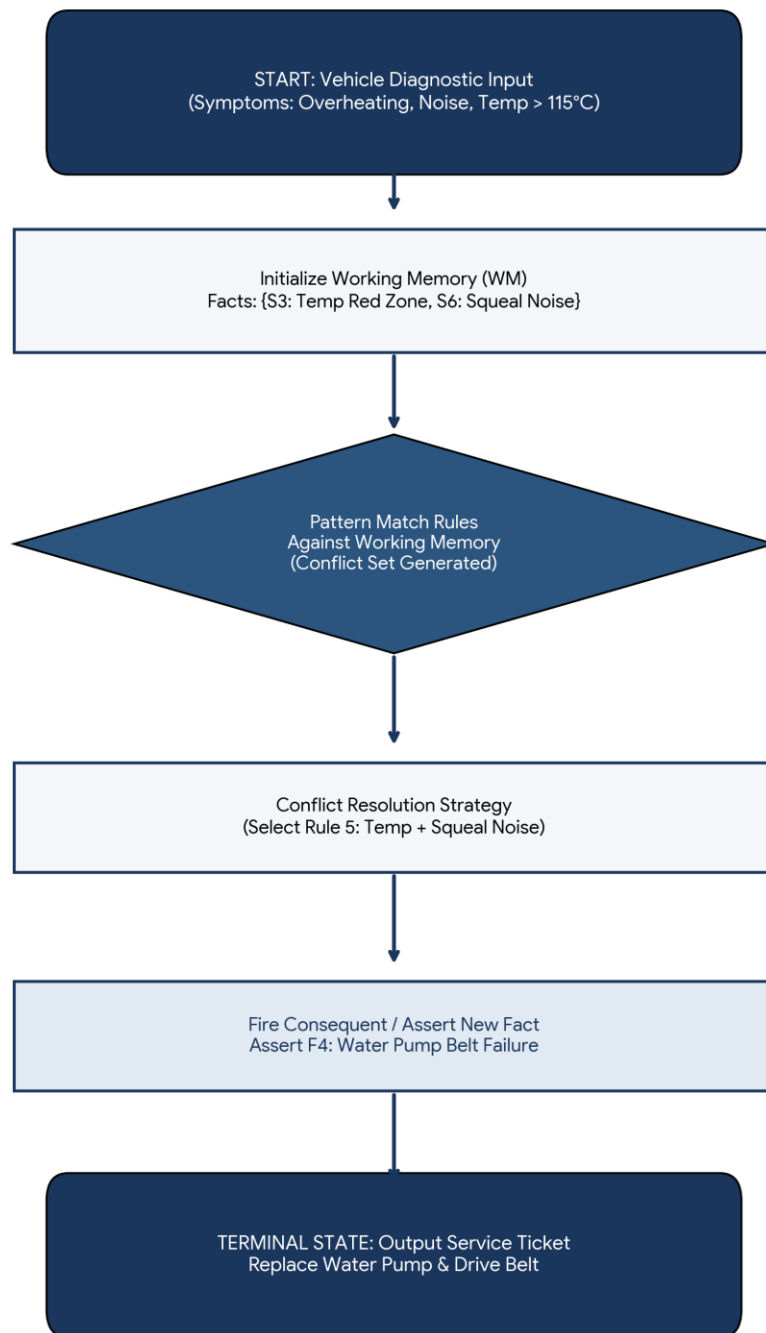


Figure 2: Forward Chaining Diagnostic Execution Flowchart.

Trace Execution Scenario: Vehicle arrives with facts $F0 = \{ S3 \text{ (Engine Temp Red Zone)}, S6 \text{ (Squealing Noise)}, \text{Coolant_Temp} = 118^{\circ}\text{C} \}$.

- **Cycle 1 - Match Phase:** Evaluates Working Memory against Rule Base. Rule 5 ($S3 \wedge S6 \rightarrow \text{Water Pump Failure}$) and Rule 4 ($S3 \wedge \text{Low Coolant} \rightarrow \text{Thermostat}$) are placed into the Conflict Set.

- **Cycle 1 - Conflict Resolution:** Specificity priority selects Rule 5 based on direct symptom matching.
- **Cycle 1 - Execution Phase:** Fires Rule 5. Asserts derived facts: I2 (Coolant Circulation Blocked) and F4 (Water Pump Failure). Working Memory updated to $F1 = F0 \cup \{ I2, F4 \}$.
- **Cycle 2 - Termination:** Rule 15 fires on F4, generating service work order and completing diagnosis.

6.2 Backward Chaining Step Trace (Goal-Driven Proving)

Trace Execution Scenario: Technician asks system: 'Is the fault F1 (Dead Battery)?' -> Goal: Prove F1.

- **Step 1 (Goal Stack: [F1]):** Look up rules concluding F1. Rule 2 matches (IF I1 $\wedge$ Voltage < 10.5V THEN F1). Goal Stack updated to [I1, Voltage < 10.5V, F1].
- **Step 2 (Goal Stack: [I1]):** Look up rules concluding I1. Rule 1 matches (IF S1 $\wedge$ S5 THEN I1). Goal Stack updated to [S1, S5, Voltage < 10.5V, F1].
- **Step 3 (Fact Verification):** Check Working Memory for S1 (Engine Cranks Slowly). S1 is FALSE (engine cranks normally). Subgoal S1 FAILS.
- **Step 4 (Backtracking & Alternate Hypothesis):** Rule 1 fails -> Subgoal I1 fails -> Goal F1 FAILS. System switches to hypothesis F4 (Water Pump Failure). Rule 5 matches. Subgoals S3 and S6 evaluated TRUE. Goal F4 PROVED successfully.

7. Exhaustive Comparative Analysis Matrix

Selecting the appropriate representation paradigm requires balancing expressiveness, inference performance, and integration complexity.

Metric / Dimension	Production Rules	Propositional Logic	First-Order Logic (FOL)	Prolog (Logic Prog.)
Expressiveness	Medium. Excellent for operational IF-THEN rules.	Low. Limited to boolean facts; no variables.	Extremely High. Quantifiers, predicates, relations.	High. Horn clause subset of FOL with predicates.
Inference Engine	Rete Algorithm, Forward/Backward Chaining.	Truth Tables, DPLL, Resolution Theorem Proving.	Unification, First-Order Resolution.	SLD Resolution with Backward Chaining.
Computational Complexity	$O(R * W)$ per cycle; scalable via Rete.	NP-Complete (SAT problem). 2^N state space.	Undecidable in general case; Semi-decidable.	$O(B^D)$ exponential worst-case, fast in practice.
Handling Uncertainty	Easy via Certainty Factors (CF) / Fuzzy	Rigid Boolean values	Requires Probabilistic	Can incorporate probabilistic Prolog

	rules.	(0 or 1).	FOL / Markov Logic.	extensions.
Suitability for Service Centers	Ideal for standard expert systems & diagnostic decision trees.	Too limited for real vehicles; useful in circuit verification.	Excellent for complex modeling, heavy computation.	Ideal for rapid prototyping of complex rule networks.

8. Strategic Recommendations & Enterprise Synthesis

Commercial automotive diagnostic software should deploy a multi-layered hybrid architecture:

- **Tier 1 - Production Rules / Prolog Core:** Use Prolog Horn clauses for rapid, real-time diagnostic symptom matching and standard trouble-code evaluation, delivering sub-millisecond execution for 90% of routine cases.
- **Tier 2 - Relational FOL Core:** Utilize First-Order Logic formalisms for complex multi-system failures involving vehicle-to-vehicle variance, custom sensor thresholds, and deep electrical wiring dependencies.
- **Tier 3 - Automated Telematics Integration:** Stream telemetry directly from vehicle OBD-II/CAN-bus networks into working memory to trigger forward chaining inference automatically before manual physical inspection.